PomoSprint — Uygulama Analiz Raporu

Tarih: 28 Eylül 2026
Kapsam: Flutter istemcisi (`lib/` , ~16.900 satır), Supabase migration'ları (`supabase/migrations/`), yardımcı scriptler (`scripts/`), landing sitesi (`landing/`)
Yöntem: Kaynak kodu okuma + `flutter analyze` . Canlı Supabase veritabanına erişilmedi (Supabase MCP bağlantısı yetkilendirilmemiş), bu yüzden veritabanı bulguları repodaki migration dosyalarına dayanıyor.

---

1. Özet

PomoSprint; Pomodoro sayacı, kategori bazlı çalışma takibi, raporlar, seri (streak) bonusu, haftalık ligler, avatar ekonomisi (kum tanesi/coin), arkadaş sistemi ve sosyal akış içeren, Supabase tabanlı bir Flutter uygulaması. TR/EN dil desteği, Google/Apple girişi, çevrimdışı seans kuyruğu, Sentry hata izleme ve bir tanıtım sitesi de var.

Kapsam olarak olgun bir ürün. Puan hesabının sunucuya taşınması, `client_uuid` ile tekrar gönderime dayanıklılık ve bir güvenlik regresyon test scripti, iyi düşünülmüş kararlar. Buna karşılık **oyun ekonomisinin tamamı istemcinin söylediğine güveniyor**, bazı RPC'ler **başka kullanıcıların e-posta adreslerini açığa çıkarıyor** ve repodaki migration'lar canlı veritabanıyla **senkron değil**. Uygulama yayına alınmadan (veya büyümeden) önce bunların düzeltilmesi gerekiyor.

Alan	Değerlendirme
Özellik kapsamı	Güçlü
Sunucu tarafı iş mantığı	İyi tasarlanmış, ama doğrulama eksik
Güvenlik	**Kritik açıklar var**
Veri senkronizasyonu / çevrimdışı	İyi fikir, uç durumlarda veri kaybı var
Kod mimarisi	Büyük "tanrı dosyaları", state yönetimi yok
Test	Neredeyse yok
Repo hijyeni	Zayıf (tek commit, izlenen build/node_modules)

---

2. Mimari

- **İstemci:** Flutter, `setState` ile durum yönetimi, tek bir singleton servis (`SupabaseService` , 1.445 satır) üzerinden tüm veri erişimi.
- **Backend:** Supabase (Postgres + Auth + RLS + `SECURITY DEFINER` RPC'ler + `pg_cron` ile haftalık lig işlemi + Edge Function'lar).
- **Ana akış:** Splash → Login/Register → Onboarding → `MainNavigationShell` (Timer, Sıralama, Raporlar, Sosyal, Profil).

- **Seans kaydı:** `TimerScreen` bitince `complete_study_session` RPC'si çağrılıyor; ağ yoksa seans `SharedPreferences`'ta kuyruğa alınıp sonra gönderiliyor.
- **Puan mantığı (sunucu):** 25 dk = 5 puan, 50 dk = 12 puan; duraklatılmışsa %20 ceza; 3/7/14 günlük seride %10/%20/%30 bonus; gün sınırı yerel saatle 04:00; lig puanı günde 16 seansla sınırlı.

En büyük dosyalar: `supabase_service.dart` (1.445), `reports_screen.dart` (1.252), `main.dart` (1.061), `history_screen.dart` (1.048).

---

3. Güçlü Yönler

1. **Puan hesabı sunucuda.** İstemci puan göndermiyor; `complete_study_session` puanı, bonusu ve seriyi kendisi hesaplıyor.
2. **Tekrar gönderime dayanıklılık.** `(user_id, client_uuid)` üzerinde unique index var, RPC başında tekrar eden istek kontrol ediliyor. Çevrimdışı kuyruk çift kayıt üretmiyor.
3. **Eşzamanlılık bilinci.** Profil satırı `FOR UPDATE` ile kilitleniyor, istemci senkronu `_isSyncingPending` bayrağıyla korunuyor.
4. **Seri mantığı iyi düşünülmüş.** Kullanıcı saat dilimi ve 04:00 gün sınırı, gece çalışanları doğru sayıyor.
5. **RLS kullanımı.** `categories`, `avatar_unlocks`, `leagues`, `league_memberships` tablolarında satır bazlı güvenlik var; lig tablolarına yazma yalnızca RPC üzerinden yapılabiliyor.
6. **Güvenlik regresyon testi.** `scripts/security_regression_test.js`, geçici hesaplarla RLS/RPC kurallarını gerçek projeye karşı test ediyor. Nadir görülen, çok değerli bir alışkanlık.
7. **Up/down migration'lar.** Her değişikliğin geri alma dosyası var.
8. **Ürün detayları:** TR/EN yerelleştirme, Google + Apple girişi, Sentry, bildirimler, Live Activities, onboarding, landing sitesi.

---

4. Bulgular

Önem seviyeleri: 🔴 Kritik · 🟠 Yüksek · 🟡 Orta · ⚪ Düşük

🔴 4.1 Seanslar gerçekten yapılmadan puan kazanılabiliyor

`complete_study_session` yalnızca şunları kontrol ediyor: süre 25 veya 50 mi, `started_at` son 24 saat içinde mi, kategori kullanıcıya mı ait, `started_at` son seansın bitişinden sonra mı. **Seansın gerçekten o kadar sürdüğü doğrulanmıyor.** `start_study_session` sunucuya bir `active_session_ends_at` yazıyor ama tamamlama RPC'si bunu hiç kontrol etmiyor; `p_is_verified` (ön planda geçen süre) de istemciden geliyor ve puanı etkilemiyor.

Senaryo: Uygulamadaki anon key ve kendi oturum token'ı olan bir kullanıcı, RPC'yi doğrudan `p_duration_minutes = 50`, `p_started_at = now() − 1 sn` ile ardı ardına çağırır. Her çağrı öncekinin bitişinden sonra başladığı için overlap kontrolünü geçer. Günde 16 seanstan sonra lig puanı duruyor ama **coin, `lifetime_points`, `total_minutes` ve genel sıralama sınırsız artıyor**; tüm avatarlar bedavaya açılabiliyor.

Çözüm: Seans başlangıcını sunucu kaydetsin (`start_study_session` bir `session_id` ve sunucu saatiyle `started_at` döndürsün). `complete_study_session` bu kaydı arasın ve `now() >= started_at + süre − küçük tolerans` şartını zorunlu kılsın. Çevrimdışı seanslar için ayrı, daha sıkı sınırlı bir yol düşünülebilir.

🔴 4.2 `profiles` tablosu migration'ı herkese okuma, kendine sınırsız yazma veriyor

`20260809033000_fix_profiles_permissions.sql` :

- `anon` rolüne (giriş yapmamış herkese) `SELECT, INSERT, UPDATE, DELETE` veriyor.
- `FOR SELECT USING (true)` : tüm profiller, e-posta dahil, herkese açık.
- `FOR UPDATE USING (auth.uid() = id)` : kolon kısıtı yok. Kullanıcı kendi `coins` , `is_premium` , `league_tier` , `current_streak` , `lifetime_points` alanlarını doğrudan değiştirebilir.

Not: `security_regression_test.js` "A, B'nin profilini okuyamaz" diye test ediyor; yani canlı veritabanı muhtemelen daha sonra elle sıkılaştırılmış. Ama **repodaki migration bu haliyle duruyor**. Migration'lar yeni bir ortama (staging, yeni proje, felaket kurtarma) uygulanırsa açık geri gelir.

Çözüm: Canlı veritabanının gerçek şemasını (`supabase db dump --schema public`) migration olarak repoya alın. `profiles` için kolon bazlı `GRANT UPDATE (name, avatar_id, show_in_social_feed, ...)` kullanın, `anon` yetkilerini kaldırın, sıralama için yalnızca gerekli kolonları döndüren bir view/RPC kullanın.

🟠 4.3 Lig sıralaması başka kullanıcıların e-postalarını sızdırıyor

`get_league_ranking(p_user_id)` `SECURITY DEFINER` ve lig üyelerinin **`email`** alanını döndürüyor. Ayrıca `p_user_id` parametresini istemciden alıyor, `auth.uid()` ile karşılaştırmıyor. Herhangi bir kullanıcı başka bir kullanıcının ID'siyle çağırarak o ligin tüm e-postalarını toplayabilir. İstemci e-postayı sadece "bu ben miyim" kontrolü için kullanıyor (`ranking_screen.dart:155`), oysa RPC zaten `is_me` döndürüyor.

Çözüm: `email` kolonunu dönüşten çıkarın, parametreyi kaldırıp `auth.uid()` kullanın. İstemcide `is_me` alanına geçin. KVKK açısından da önemli.

🟠 4.4 Abonelikten çıkma sayfasında XSS ve kimlik doğrulamasız çıkış

`landing/unsubscribe.html:40` , URL'deki `email` parametresini `innerHTML` ile sayfaya basıyor. `?email=<img src=x onerror=...>` gibi bir bağlantı sitenizin alan adında script çalıştırır (yansıtılmış XSS). Ayrıca çıkış işlemi yalnızca e-postayla yapılıyor. Herhangi biri, herhangi birini listeden çıkarabilir.

Çözüm: `textContent` kullanın. Bağlantıya sunucuda üretilmiş imzalı bir token ekleyin (`?token=...`), Edge Function'da doğrulayın.

🟡 4.5 Çevrimdışı seanslar kalıcı olarak takılabiliyor / kaybolabiliyor

`syncPendingSessions` hata alan her seansı kuyrukta tutuyor. Ama bazı hatalar geçici değil:

- `started_at` 24 saatten eskiyse sunucu `RAISE EXCEPTION` yapıyor. Seans **asla gönderilemez** ve her açılışta yeniden denenir. 24 saatten uzun çevrimdışı kalan kullanıcının çalışması kaybolur.
- Eski formattaki kayıtlar için yedek kategori `00000000-...` ; sunucu bunu reddeder, aynı sonsuz döngü oluşur.

Çözüm: Hataları "geçici" (ağ) ve "kalıcı" (doğrulama) olarak ayırın; kalıcı olanları kuyruktan çıkarıp kullanıcıya bildirin. Çevrimdışı seanslar için 24 saat sınırını gözden geçirin (ör. ödül vermeden sadece geçmişe yazmak).

🟡 4.6 Kategori limitleri istemci tarafında aşılabiliyor, senkron çakışıyor

- `create_category` RPC'si 8 (ücretsiz) / 15 (premium) limitini uyguluyor. Ama `insertCategory` ve `upsertCategories` doğrudan `categories` tablosuna yazıyor, limit atlanıyor. `unarchiveCategory` içindeki premium kontrolü de istemcide.
- `main.dart:378` ve `splash_screen.dart:176` her açılışta yerel önbellekteki kategorileri buluta **upsert** ediyor. Başka cihazda silinmiş veya düzenlenmiş kategoriler eski yerel sürümle ezilir ya da geri gelir.
- `upsertCategories` ve `updateCategoryOrders` kategori başına ayrı istek atıyor (N istek).

Çözüm: Tüm kategori yazmalarını RPC'lere taşıyın; limiti tablo üzerinde bir trigger ile garanti edin. Kaynak olarak sunucuyu kabul edin: açılışta buluttan çekin, yerel önbelleği yalnızca çevrimdışı gösterim için kullanın. Toplu upsert tek istekle yapılabilir.

🟡 4.7 Migration'lar yeni bir ortamı kuramıyor; dokümantasyon eskimiş

Migration'lar `profiles`, `study_sessions`, `leagues`, `friendships` gibi tabloları **oluşturmadan** değiştiriyor (başlangıç şeması repoda yok). Canlı veritabanı da dosyalardan farklı (bkz. 4.2). `PROJECT_CONTEXT.md` arkadaşlık özelliğinin silindiğini ve `increment_user_stats` RPC'sinin kullanıldığını söylüyor; ikisi de artık doğru değil. Yapay zekâ asistanları bu dosyayı bağlam olarak okuduğu için yanlış yönlendirilebilir.

Çözüm: Canlı şemadan bir "baseline" migration üretin, `PROJECT_CONTEXT.md`'yi güncelleyin.

🟡 4.8 İstemci ve sunucu puan tabloları farklı

`lib/utils/points.dart`: 50 dk için 10 puan. Sunucu: 12 puan. Fonksiyon şu an hiçbir yerde çağrılmıyor ama yanıltıcı. İleride biri "tahmini puan" göstermek için kullanırsa kullanıcıya yanlış bilgi verir.

Çözüm: Dosyayı silin veya değerleri `get_points_config` RPC'sinden okuyun.

🟡 4.9 `search_path` ayarlanmamış `SECURITY DEFINER` fonksiyonlar

`start_study_session`, `cancel_study_session`, `get_live_friends`, `get_friends_feed`, `give_default_avatars` ve `process_weekly_leagues`'in son sürümünde `SET search_path` yok. Supabase linter bunu uyarı olarak işaretler; şema gölgeleme (schema shadowing) saldırılarına kapı aralar.

Çözüm: Her `SECURITY DEFINER` fonksiyona `SET search_path = public` ekleyin ve `REVOKE EXECUTE ... FROM anon` uygulayın.

🟡 4.10 Release derlemesi debug anahtarıyla imzalanabiliyor

`android/app/build.gradle.kts:57-60`: `key.properties` yoksa release derlemesi sessizce **debug** anahtarıyla imzalanıyor. Yanlışlıkla Play Store'a yüklenmeye çalışılırsa reddedilir. Daha kötüsü, bu APK dağıtılırsa güncellemeler imza uyuşmazlığına düşer.

Çözüm: Keystore yoksa release derlemesini hata ile durdurun.

🟡 4.11 Test yok denecek kadar az

- `test/widget_test.dart` Flutter'ın varsayılan "sayaç" testi; bu uygulamada **başarısız olur**.
- `test/register_test.dart` sadece `expect(true, isTrue)`.
- Puan, seri, çevrimdışı kuyruk gibi kritik mantığın birim testi yok.

Çözüm: Önce SQL tarafı: `complete_study_session` için pgTAP veya mevcut Node scriptine senaryolar ekleyin. Sonra `syncPendingSessions` ve kategori senkronu için Dart birim testleri yazın.

⚪ 4.12 Kod yapısı ve bakım

- **Tanrı dosyaları:** `SupabaseService` auth, seans, kategori, arkadaş, avatar, lig işlerinin hepsini yapıyor. `main.dart` 1.000+ satır ve iş mantığı içeriyor. Alan bazlı servislere (AuthRepository, SessionRepository, CategoryRepository…) bölünmeli.
- **State yönetimi yok:** Tüm durum `setState` + callback'lerle ekranlar arasında taşınıyor. Riverpod veya Provider gibi bir çözüm test edilebilirliği ciddi artırır.
- **Sayaç kayması:** `Timer.periodic` ile saniye saniye geri sayılıyor, arka plandaki süre lifecycle olaylarıyla telafi ediliyor. Kalan süreyi her tick'te `bitiş_zamanı − now()` olarak hesaplamak daha sağlam.
- **Ölü kod:** `DatabaseInspectorScreen` ve `DatabaseHelper` (sqflite) hiçbir yerden açılmıyor; `insertStudySession` RPC'yi atlayıp tabloya doğrudan yazıyor ve kullanılmıyor. Kaldırılmalı.
- **Sabit saat dilimi:** Lig haftası `Europe/Istanbul`'a sabit. Uluslararası kullanıcılar için hafta farklı bir saatte biter.
- **`flutter analyze`:** 14 sorun (4 uyarı: kullanılmayan import, kullanılmayan alan, ölü kod, gereksiz null kontrolü; 10 bilgi: kullanımdan kalkmış API'ler, `BuildContext`'in async boşluktan sonra kullanımı).
- **Kullanıcı verisi anahtarı:** Bekleyen seanslar ve bazı tercihler e-posta adresine göre saklanıyor; kullanıcı ID'si daha kararlı olur.

⚪ 4.13 Repo hijyeni

- Sadece tek bir "Initial commit" var ve 105 dosya commit edilmemiş değişiklik içeriyor. Değişiklik geçmişi ve geri dönüş imkânı yok.
- `landing/node_modules/` ve `android/build/reports/` git'te izleniyor.
- `README.md` Flutter'ın varsayılan şablonu.
- Olumlu: `.env` ve `scripts/.env` git'te değil. Uygulamaya gömülen `.env` yalnızca herkese açık anahtarları (Supabase anon, Sentry DSN) içeriyor. Service role anahtarı yalnızca `scripts/.env`'de.

---

5. Önceliklendirilmiş Eylem Planı

#	İş	Önem	Tahmini efor
1	Seans süresini sunucuda doğrula (4.1)	🔴	1–2 gün
2	Canlı şemayı dump edip migration'ları senkronla; `profiles` yetkilerini kolon bazlı yap (4.2, 4.7)	🔴	1 gün
3	`get_league_ranking`'den e-postayı çıkar, `auth.uid()` kullan (4.3)	🟠	1–2 saat
4	`unsubscribe.html` XSS düzeltmesi + imzalı token (4.4)	🟠	2–3 saat
5	Tüm `SECURITY DEFINER` fonksiyonlara `search_path` + `REVOKE` (4.9)	🟡	1 saat
6	Çevrimdışı kuyrukta kalıcı/geçici hata ayrımı (4.5)	🟡	yarım gün
7	Kategori yazmalarını RPC'ye taşı, senkronu sunucu öncelikli yap (4.6)	🟡	1 gün
8	Release imzalama fallback'ini kaldır (4.10)	🟡	15 dk
9	Git düzeni: `.gitignore`'u uygula, anlamlı commit'lere böl (4.13)	⚪	1 saat
10	Kritik mantık için testler (4.11)	🟡	sürekli
11	`SupabaseService` ve `main.dart`'ı böl, state yönetimi ekle (4.12)	⚪	kademeli

6. Sonuç

Yapay zekâ ile geliştirilmiş bir proje için kapsam ve bazı mimari kararlar (sunucu tarafı puan, tekrar gönderime dayanıklılık, regresyon testi) beklenenin üzerinde. Zayıf noktalar da bu tür projelerin tipik zayıf noktaları: **istemciye fazla güvenen sunucu mantığı**, **yeniden üretilemeyen veritabanı şeması**, **test eksikliği** ve **büyüyen tek dosyalar**. İlk dört madde birkaç günlük işle kapatılabilir ve uygulamanın güvenlik seviyesini belirgin şekilde yükseltir.